COURSE OUTLINE

Gennoor Tech
TRAIN. INNOVATE. BUILD.

Configuration Comparison Automation with Claude

Two platforms, one verified difference report, before the change window opens

3 days (8 hours per day) | Intermediate | Instructor-led, hands-on labs | Capstone project

About this course

Who should attend

Enterprise IT and network operations teams, load balancer and ADC engineers, cloud migration and platform engineers, infrastructure automation leads.

Prerequisites

- Working knowledge of load balancers or ADCs (F5, NetScaler, Radware or similar) and basic cloud networking
- Comfort with the command line, REST APIs and JSON
- Basic Python (reading and editing scripts)
- Familiarity with Git; prior use of Claude or an AI coding assistant is helpful, not required

What you will be able to do

- Plan a configuration comparison for cloud-to-cloud, physical-to-cloud and cloud-to-physical migrations, including what must be proven and what cannot be compared
- Collect running configuration from F5 (iControl REST and tmsh), Citrix NetScaler (NITRO API), Radware Alteon, other load balancers (Anaya LB), Cisco APIC and Terraform state with read-only credentials
- Normalise every platform into one canonical Pydantic model with provenance and explicit unknowns
- Use Claude Code, the Claude Agent SDK and MCP servers so that Claude plans and explains while deterministic Python computes the diff
- Generate severity-classified difference reports, Claude-written explanations and remediation steps grounded in the report

For more info - https://gennoor.com

- Gate every consequential action behind GitHub pull request review and a ServiceNow change record, with a full audit trail of prompts, tool calls and decisions

Tools and technologies

Claude Code · Claude Agent SDK and MCP · Python · Pydantic · F5 iControl REST / tmsh · NetScaler NITRO API · Radware Alteon · Cisco APIC REST API · Terraform · GitHub · GitHub Actions · ServiceNow

Table of contents

DAY 1 4 modules

MODULE 1

Configuration comparison as a migration discipline

- The three directions: cloud-to-cloud, physical-to-cloud, cloud-to-physical, and what each must prove before cut-over
- Why a raw text diff between an F5 and a NetScaler configuration is worthless; semantic objects versus syntax
- Scoping the comparison: virtual servers, pools, monitors, persistence, SSL profiles, policies, routing and NAT
- The course pattern: Claude plans, drafts and explains; deterministic code executes; a person approves; everything is logged

HANDS-ON LAB
Take a sample physical F5 configuration and a hand-built cloud NetScaler configuration and write, by hand, the comparison plan and object checklist that the pipeline must later automate.

DELIVERABLE
Comparison plan template and object checklist.

Gennoor Tech · INNOVATE · BUILD · https://gennoor.com · for more info

MODULE 2

Setting up Claude Code, the Agent SDK and MCP for the workflow

- Installing Claude Code and configuring a project with CLAUDE.md conventions for the comparison repository
- Claude Agent SDK basics: sessions, tool definitions, structured JSON outputs and when to use the SDK instead of the interactive CLI
- Building a minimal MCP server that exposes read-only collector tools (get_virtual_servers, get_pools) to Claude
- Secrets handling: credentials in environment variables and vaults, never in prompts or repositories

HANDS-ON LAB

Stand up the workbench, register the read-only MCP server, and have Claude list the virtual servers on the lab F5 through tool use without ever seeing a password.

DELIVERABLE

Configured workbench with a working read-only MCP collector.

MODULE 3

Collecting configuration from every platform

- F5: iControl REST (/mgmt/tm/ltm) and tmsh list output; Citrix NetScaler: NITRO API config resources
- Radware Alteon via Cyber Controller and CLI exports; other load balancers (Anaya LB) through CLI capture adapters
- Cisco APIC REST API for fabric objects and Terraform state (terraform show -json) for cloud-side resources
- Collector contract: idempotent, read-only, timestamped, raw payload retained as evidence

HANDS-ON LAB

Write Python collectors for F5 iControl REST and NetScaler NITRO against the lab devices, then Radware Alteon CLI export and Cisco APIC REST collectors against recorded API fixtures, with Claude drafting the code and you reviewing every diff.

DELIVERABLE

Collector library with raw evidence capture for four platform families (F5, NetScaler, Radware Alteon, Cisco APIC).

Gennoor Tech · AI · INNOVATE · BUILD · For more info - https://gennoor.com

MODULE 4

Normalising into a canonical configuration model

- Designing the canonical Pydantic model: VirtualServer, Pool, Member, Monitor, Persistence, SSLProfile, Policy
- Mapping tables per platform and the rule that unmapped fields are marked unknown, never dropped silently
- Provenance on every field: source platform, object name, collection time, raw reference
- Validation tests for round-tripping fixtures through the model

HANDS-ON LAB
Build the normaliser for F5 and NetScaler, run both lab configurations through it, and inspect the unknowns list with Claude explaining each one.

DELIVERABLE
Canonical model, mapping tables and normalisation test suite.

DAY 2 3 modules

MODULE 5

Migration planning with Claude

- Prompting Claude with the source inventory and target platform capabilities to draft a migration plan as structured JSON
- Required fields: object mapping, feature gap, dependency order, risk, confidence, owner, and citation to the inventory
- Handling incomplete or contradictory inventories: the plan must say what cannot be mapped
- Reviewing and versioning plans in Git as the first human checkpoint

HANDS-ON LAB
Generate a physical-to-cloud plan for the lab inventory, then feed Claude a deliberately contradictory inventory and verify it flags the conflict instead of inventing a mapping.

DELIVERABLE
Migration planning agent with JSON schema and review checklist.

Gennoor Tech INNOVATE. BUILD. For more info, https://gennoor.com

MODULE 6

The deterministic diff engine

- Field-by-field comparison of canonical models in Python; ordering, set semantics and tolerated differences
- Severity classification rules: blocking, warning, informational, and platform-specific exemptions
- Machine-readable diff report (JSON) plus a human summary; the engine never calls a model
- Unit tests with pytest, including seeded mismatches that must always be caught

HANDS-ON LAB
Implement the diff engine, seed a hidden persistence timeout mismatch into the target configuration, and prove the test suite catches it.

DELIVERABLE
Tested diff engine producing severity-classified JSON reports.

MODULE 7

Claude explains the differences

- Giving Claude the diff report through tool use (get_diff, get_object) rather than pasting whole configurations
- Structured explanation output: what differs, why it matters for traffic, recommended remediation, confidence, citation to the diff line
- Drift summaries for operations managers and remediation steps for engineers from the same report
- Guarding against hallucinated objects: every referenced object must exist in the report

HANDS-ON LAB
Build the explanation agent with the Agent SDK, run it on the day's diff report, and verify every statement traces to a report line.

DELIVERABLE
Explanation agent and remediation report template.

DAY 3 3 modules

Gemnoor Tech PRIVATE LIMITED
For more info- https://gemnoor.com

MODULE 8

Guardrails, approvals and audit for configuration comparison

- Read-only credentials for all collectors; no write capability exposed to Claude through MCP
- Opening a ServiceNow change record with the Table API and attaching the report; Flow Designer approval routing to a named approver
- Publishing each comparison as a GitHub pull request with the report, evidence and plan for review
- Audit log of prompts, tool calls, model outputs and human decisions, retained alongside the raw evidence

HANDS-ON LAB

Wire the pipeline to create a ServiceNow change via the Table API and a GitHub pull request, then attempt an unapproved remediation and confirm it is blocked and logged.

DELIVERABLE

Approval and audit workflow with blocked-action evidence.

MODULE 9

Scheduling, Terraform plan checks and GitHub Actions

- Running the comparison on a schedule in GitHub Actions and on pull requests that change Terraform code
- Comparing terraform plan output against live collected state before apply; flagging drift on both sides
- Notifications and report publishing: PR comments, ServiceNow updates, archived reports
- Operating the pipeline: failures, retries, fixture refresh and runbook

HANDS-ON LAB

Build the GitHub Actions workflow that runs the comparison, compares a Terraform plan against the live cloud load balancer, and comments the result on the pull request.

DELIVERABLE

Scheduled comparison pipeline with Terraform drift check and runbook.

GenNoorTech
AI TRAINING INNOVATE. BUILD.
For more info: https://gennoor.com

MODULE 10

Capstone: end-to-end physical-to-cloud comparison

- Collect, normalise, plan, diff and explain a physical F5 to cloud NetScaler migration end to end
- Produce the approval package: plan, severity report, Claude explanation, evidence, ServiceNow change, pull request
- Demo and acceptance review against the module one checklist

HANDS-ON LAB

Run the full pipeline on the lab physical F5 and cloud NetScaler, produce the approval package with ServiceNow change and pull request, and present it for acceptance review against the module one checklist.

DELIVERABLE

Complete comparison pipeline and approval package.

Capstone

Capstone project

A repeatable pipeline that collects configuration from two platforms, normalises both into a canonical model, has Claude draft the migration plan and explain a deterministic diff, and delivers an approval package through a GitHub pull request and a ServiceNow change record with a full audit trail and no change applied without approval.

Assessment

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs

Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

Engineering principles used throughout

- Model output is a proposal, not a fact: every finding and generated change is validated by deterministic checks and tests.
- Authorization lives outside the model: tool access is enforced by application code, scoped credentials and policy.
- Human approval for consequential actions: deploying a mitigation, changing production configuration, rotating credentials.
- Decisions, evidence, model and tool versions, approvals and outcomes are logged without recording secrets.
- A model adapter and a tool adapter layer keep every build provider-agnostic and enterprise-portable.
- Quality is measured: false positives, false negatives, reliability, latency, cost and safe failure, not just a demo.

© Gennoor Tech · Train. Innovate. Build.

For more info - https://gennoor.com